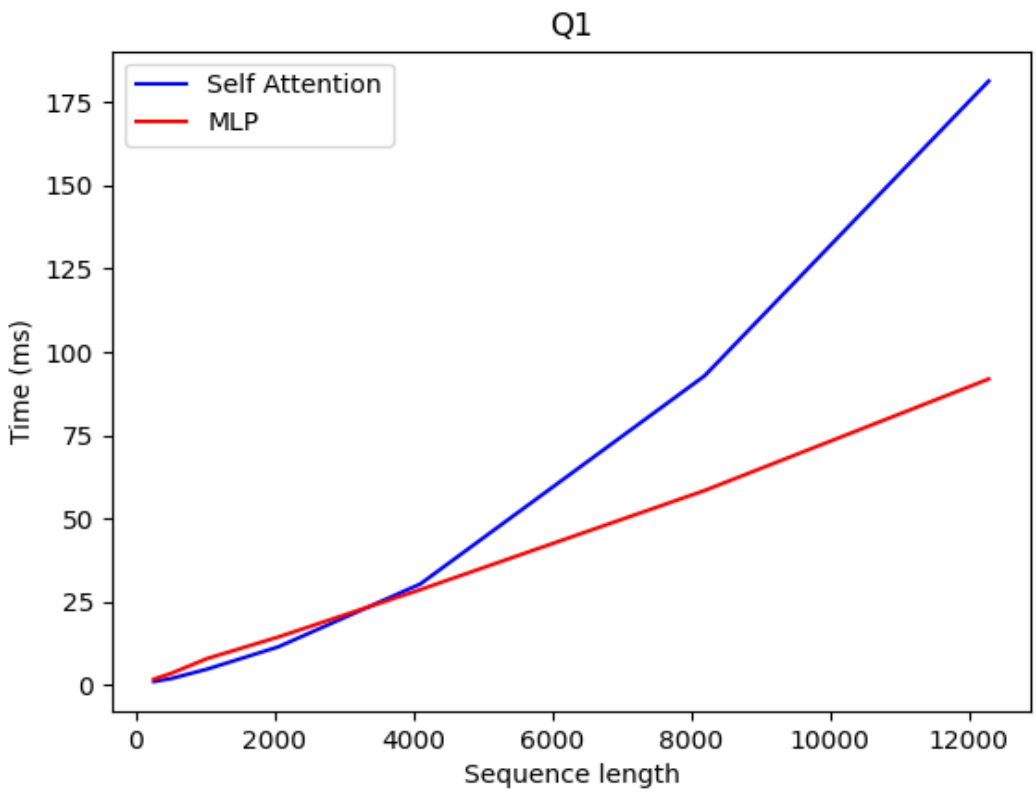
Lab 4 – Part 2 Report

Mohammed Wazir Adain

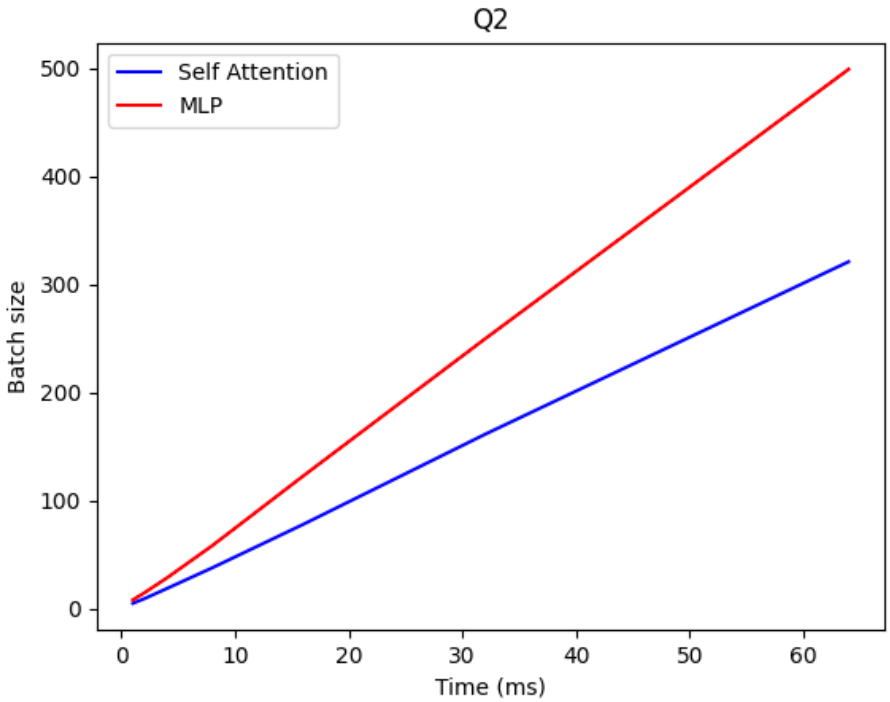
Q1. Batch size = 1



Explanation:

MLP scales linearly, Self attention scales non linearly (quadratically). Reasons:
MLP – O(N) operation
Self attention – O(N^2) operation, as each token has to attend all the tokens before it.
Time taken to move the vectors from HBM to the SMs also increase with the sequence length.

Q2. Seq_len = 1024



MLP scales linearly with batch size.

Linear Latency Scaling: The constant upward slope indicates that every additional element in the batch adds a fixed amount of data-movement overhead. We are not yet "hiding" the cost of memory access behind computation.

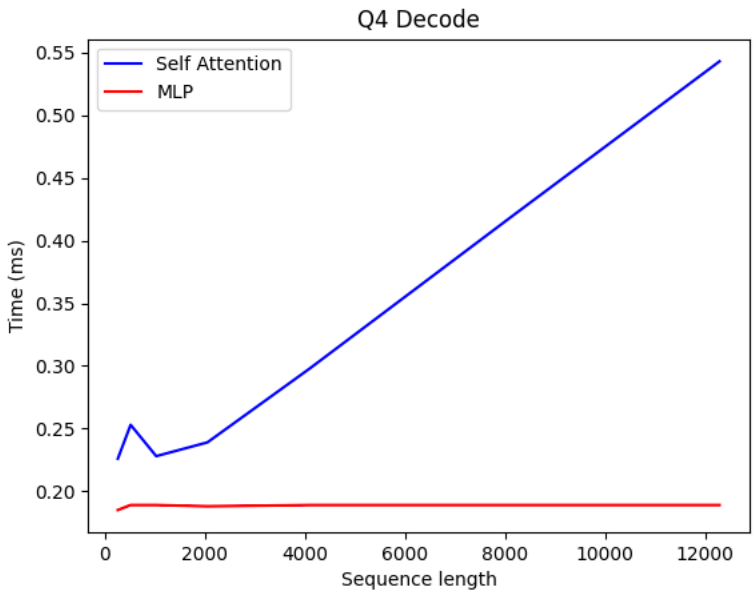
The **MLP** layer (red) scales more aggressively (steeper slope) than **Self-Attention** (blue). This is because MLPs typically involve larger weight matrices that must be loaded entirely for each forward pass, whereas Self-Attention's memory footprint at these scales is dominated by the KV cache.

Q3.
bench_decode()

```
def bench_decode(batch: int, seq_len: int, device: torch.device) -> None:
    """
    Profile the Decode stage of a single Llama decoder block.

    During decode, the model receives ONE new token (seq_len=1) together with
    a KV-cache that holds the key/value tensors for all *previous* tokens.
    """
    model = LlamaDecoderBlock().to(device).eval()

    # The single new token's hidden state: shape (batch, 1, hidden_dim)
    hidden  = torch.randn(batch, 1, HIDDEN_DIM, device=device)
    # Position of the current token (0-indexed)
    pos_ids = torch.tensor([[seq_len - 1]], device=device).expand(batch, -1)

    ########## Solution Block ##########
    # Declare random PyTorch tensors that match the shapes of the KV cache.
    #
    # The KV cache holds key and value tensors for all *previous* tokens.
    # At decode step for token at position (seq_len - 1), there are
    # (seq_len - 1) cached tokens.
    #
    # Required shapes:
    #   past_k : (batch, NUM_HEADS, seq_len - 1, HEAD_DIM)
    #   past_v : (batch, NUM_HEADS, seq_len - 1, HEAD_DIM)
    #
    # Store them as a tuple:  past_key_value = (past_k, past_v)

    past_k = torch.randn(batch, NUM_HEADS, seq_len - 1, HEAD_DIM, device=device)
    past_v = torch.randn(batch, NUM_HEADS, seq_len - 1, HEAD_DIM, device=device)

    past_key_value = (past_k, past_v)   # ← replace with real tensors

    ########## End Solution Block ##########

    if past_key_value is None:
        raise RuntimeError(
            "past_key_value is not implemented. "
            "Fill in the Solution Block in bench_decode()."
        )

    with torch.no_grad():
        # --- Total ---
        _, total_ms = _timed(lambda: model(hidden, pos_ids, past_key_value),
device)

        # --- Self-attention only ---
        normed = model.input_layernorm(hidden)
```

```
        (attn_out, _), attn_ms = _timed(
            lambda: model.self_attn(normed, pos_ids, past_key_value), device
        )

        # --- MLP only ---
        post_attn   = hidden + attn_out
        normed_post = model.post_attention_layernorm(post_attn)
        _, mlp_ms = _timed(lambda: model.mlp(normed_post), device)

    misc_ms = max(total_ms - attn_ms - mlp_ms, 0.0)

    print("Decode Stage Time Results (ms)")
    print(f"Input size: Batch={batch}, Seq_len={seq_len}, Hidden_dim={HIDDEN_DIM}")
    print(f"{'Self-Attn':>10}, {'MLPs':>10}, {'Misc':>10}, {'Total':>10}")
    print(f"{attn_ms:10.3f}, {mlp_ms:10.3f}, {misc_ms:10.3f}, {total_ms:10.3f}")


```

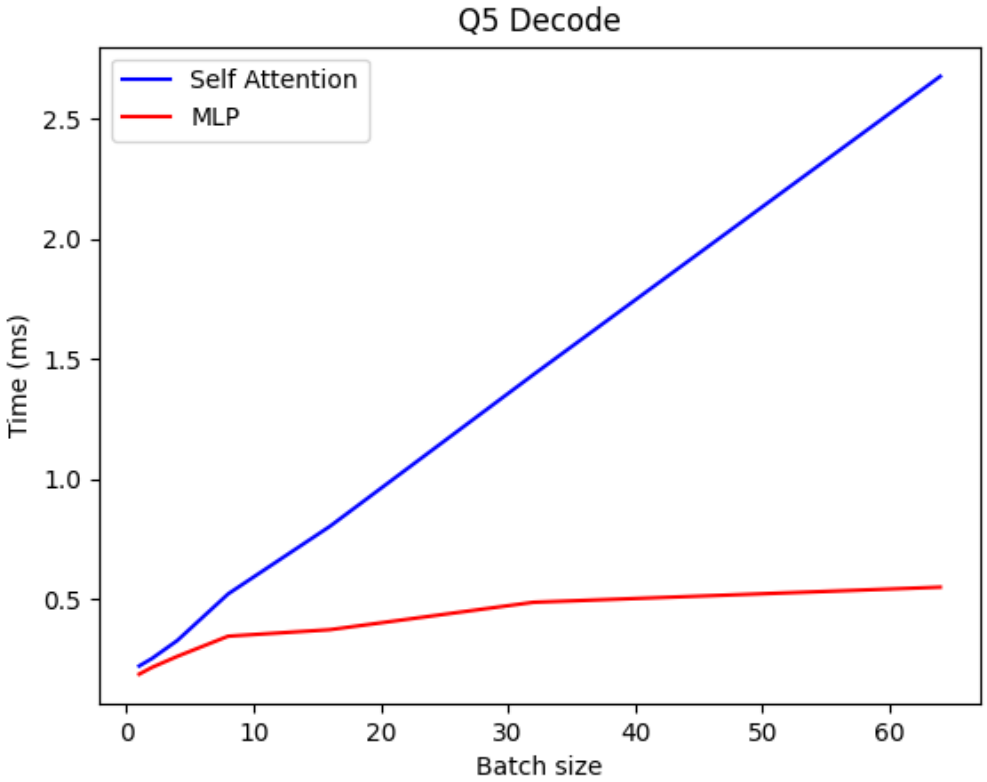
Q4.
Batch = 1



Time taken for MLP decode is independent of sequence length, as only the new token is processed – O(1) time in MLP and for self-attention, the new token attends to the older tokens through the K and V values that are cached.

As the sequence length increases, the time taken to load the KV values from HBM increases, hence the increase in time.

Q5. Seq_len = 1024



MLP - Sub-linear / Flattening - Weight Reuse: The same model weights are applied to all tokens in the batch, allowing compute units to hide memory latency.
Self Attention – Linear - Data Movement: Unique KV cache entries must be fetched for every batch member; the memory bus is the primary bottleneck.

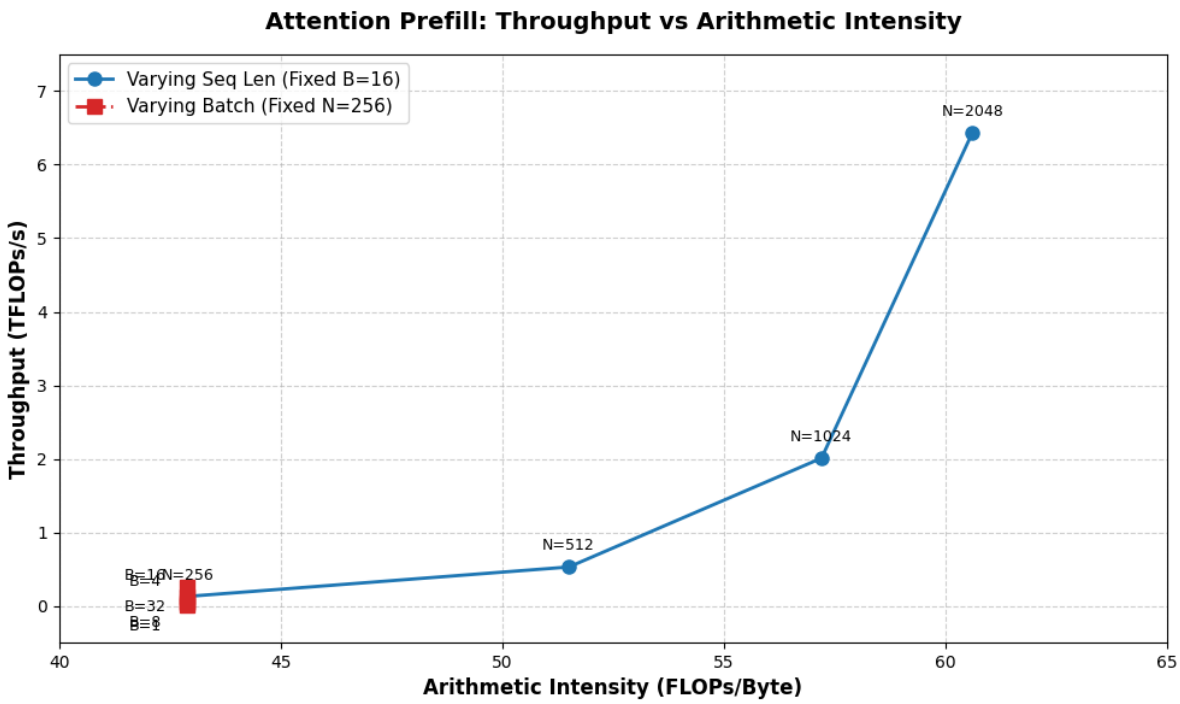
Q6. What is the dominant operator in Prefill vs. Decode when sequence length is long?

As the sequence length increases, prefill becomes the dominant operator. This is because, Self attention is an O(N^2) operation

Q7.
Throughput vs AI (Arithmetic Intensity)

Seq Len (N)	Time (ms)	Compute (GFLOPs)	AI (FLOPs/Byte)	Throughput (TFLOPs/s)
256	129.61	17.28	42.9	0.13
512	131.37	69.12	51.5	0.53
1024	137.57	276.48	57.2	2.01
2048	171.87	1105.9	60.6	6.43

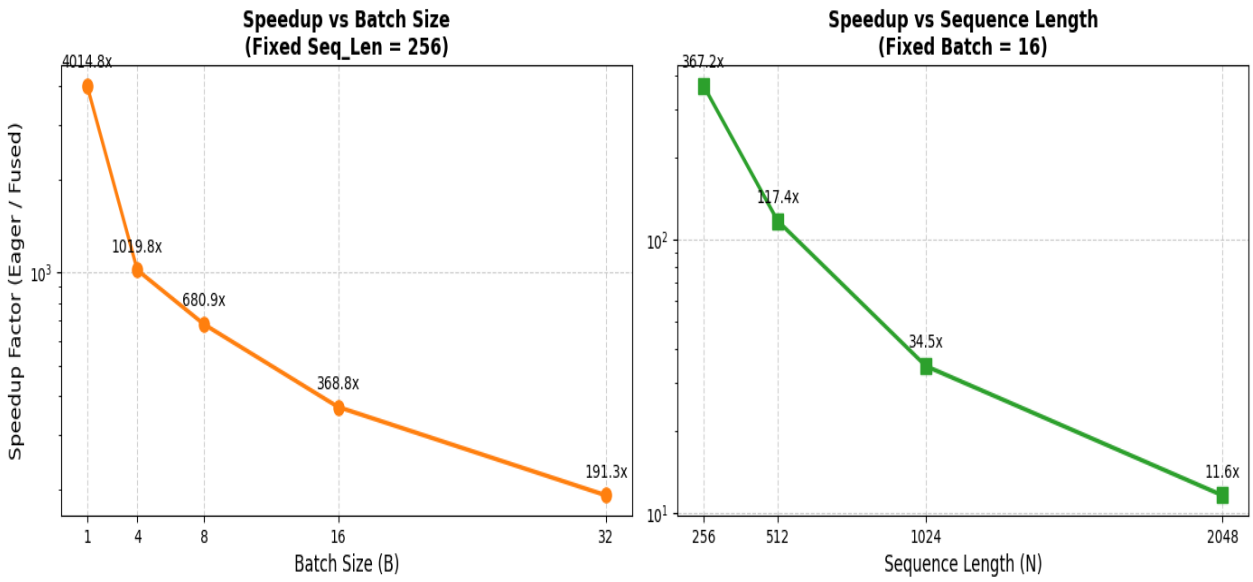
Batch (B)	Time (ms)	Compute (GFLOPs)	AI (FLOPs/Byte)	Throughput (TFLOPs/s)
1	124.46	1.08	42.9	0.008
4	107.08	4.32	42.9	0.040
8	129.36	8.64	42.9	0.066
16	129.80	17.28	42.9	0.133
32	130.64	34.56	42.9	0.264



Explanation:
Sequence Length Sweep (Increasing AI) As sequence length N increases, the O(N^2) math operations begin to outpace the O(Nd) memory loads.

The Batch Size Sweep (Static AI) This is where standard eager attention falls flat. Because the arithmetic intensity is locked at 42.9 regardless of batch size, the operation is fundamentally memory-bandwidth bound. Throughput scales linearly purely because you are amortizing fixed overheads, but you hit a hard ceiling quickly.

Q8.



Seq Len (N)	Compute (GFLOPs)	Eager (ms)	Eager (TFLOPs/s)	Fused (ms)	Fused (TFLOPs/s)	Speedup
256	17.28	129.617	0.133	0.353	48.95	**367.2x**
512	69.12	131.372	0.526	1.119	61.77	**117.4x**
1024	276.48	137.570	2.010	3.986	69.36	**34.5x**
2048	1105.92	171.870	6.435	14.762	74.92	**11.6x**
4096	4423.68	*OOM*	-	57.175	77.37	-
8192	17694.72	*OOM*	-	224.517	78.81	-
12288	39813.12	*OOM*	-	505.265	78.80	-
16384	70778.88	*OOM*	-	902.383	78.44	-

Batch (B)	Compute (GFLOPs)	Eager (ms)	Eager (TFLOPs/s)	Fused (ms)	Fused (TFLOPs/s)	Speedup
1	1.08	124.460	0.009	0.031	34.84	**4014.8x**
4	4.32	107.081	0.040	0.105	41.14	**1019.8x**
8	8.64	129.362	0.067	0.190	45.47	**680.9x**
16	17.28	129.801	0.133	0.352	49.09	**368.8x**
32	34.56	130.647	0.265	0.683	50.60	**191.3x**

Explanation:
Eager Attention AI (Memory Bound): The standard implementation writes the N x N attention matrix to HBM. It reads and writes to the HBM for every operation, thereby reducing the Arithmetic intensity drastically.

Fused Attention AI (Compute Bound): FlashAttention uses kernel fusion and tiling to keep the N x N matrix in SRAM. It only reads Q, K, V from HBM once and writes the final output O once. The memory traffic drops drastically to roughly 8 x N x d bytes per head.

Thus, Instead of hitting a ceiling, the arithmetic intensity grows linearly with sequence length.

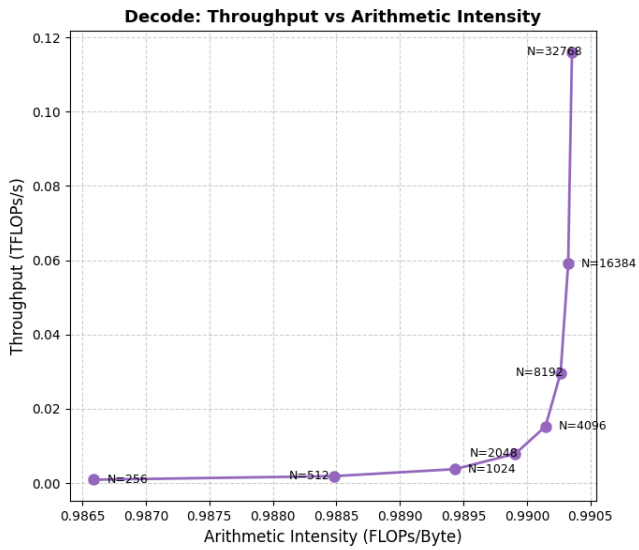
Q9. Bench_attn_decode

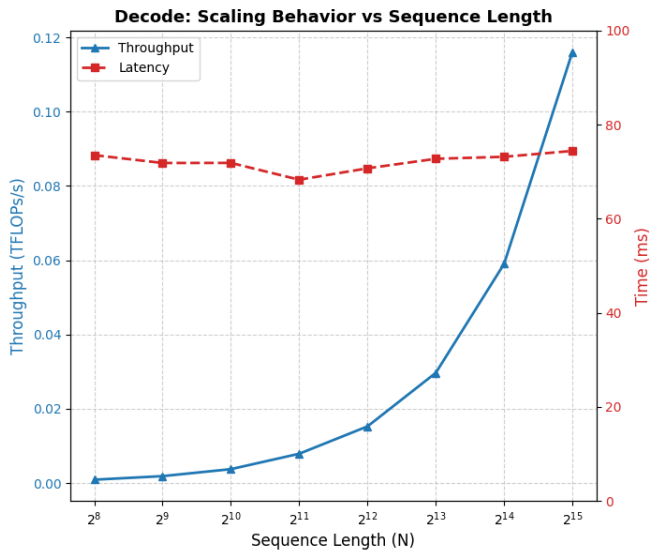
```
def bench_attn_decode(
    batch:      int,
    seq_len:    int,
    num_heads:  int,
    head_dim:   int,
    fused:      bool,
    device:     torch.device,
) -> None:
    """
    Benchmark self-attention for the Decode stage.

    During decode, only ONE new query token is computed; the keys and values
    for all previous (seq_len) tokens come from the KV cache.

    You must implement the Solution Block below to set up the tensors and
    compute each attention step with timing.
    """

    scale = head_dim ** -0.5

    print("Benchmarking for Attention decode stage in ms")
    print(f"Input size: Batch={batch}, Seq_len={seq_len}, Num_heads={num_heads},
Embed_dim={head_dim}")

    ########## Solution Block — Decode Attention ##########
    # Implement decode-stage attention with KV-cache tensors.
    #
    # In the decode stage:
    #   - The model generates ONE new token at a time.
    #   - Q_new is the query for that single new token.
    #   - K_cache and V_cache hold keys/values for all previous tokens.
    #   - No causal mask is needed (the single query attends to ALL past
positions).
    #
    # Step 0: Declare random tensors with the correct shapes:
    #   Q_new   : (batch, num_heads,       1, head_dim)  ← single query
    #   K_cache : (batch, num_heads, seq_len, head_dim)  ← cached keys
    #   V_cache : (batch, num_heads, seq_len, head_dim)  ← cached values
    #   Then set: Q, K, V = Q_new, K_cache, V_cache
    #
    # Then implement and time each step (same pattern as prefill, but no masking).

    # 0. Declare KV-cache tensors
    Q_new = torch.randn(batch, num_heads, 1, head_dim, device=device)
    K_cache = torch.randn(batch, num_heads, seq_len, head_dim, device=device)
    V_cache = torch.randn(batch, num_heads, seq_len, head_dim, device=device)
    Q = Q_new   # ← implement
    K = K_cache   # ← implement
```

```
    V = V_cache  # ← implement

    with torch.no_grad():
        # 1. Q·K^T — shape (batch, num_heads, 1, seq_len)
        _sync(device)
        t0 = time.perf_counter()
        scores = torch.matmul(Q, K.transpose(-2, -1)) * scale   # ← implement: Q @
K^T * scale
        _sync(device)
        qkt_ms = (time.perf_counter() - t0) * 1e3

        # 2. Softmax — normalize over the last dimension
        _sync(device)
        t0 = time.perf_counter()
        weights = F.softmax(scores, dim=-1)  # ← implement: softmax of scores
        _sync(device)
        softmax_ms = (time.perf_counter() - t0) * 1e3

        # 3. Attention · V — shape (batch, num_heads, 1, head_dim)
        _sync(device)
        t0 = time.perf_counter()
        output = torch.matmul(weights, V)   # ← implement: weights @ V
        _sync(device)
        attn_v_ms = (time.perf_counter() - t0) * 1e3

    ########## End Solution Block — Decode Attention ##########

    if Q is None or K is None or V is None:
        raise RuntimeError(
            "Decode attention not implemented. "
            "Fill in the Solution Block in bench_attn_decode()."
        )

    if fused:
        with torch.no_grad():
            _, total_ms = _timed(
                lambda: F.scaled_dot_product_attention(Q, K, V, is_causal=False),
                device,
            )
        print(f"{'Total':>8}")
        print(f"{total_ms:8.3f}")
        return

    total_ms = qkt_ms + softmax_ms + attn_v_ms

    print(f"{'Q_K^T':>8}, {'Softmax':>8}, {'Attn_V':>8}, {'Total':>8}")
    print(f"{qkt_ms:8.3f}, {softmax_ms:8.3f}, {attn_v_ms:8.3f}, {total_ms:8.3f}")


```

Q10.

Seq Len (N)	Compute (GFLOPs)	AI (FLOPs/Byte)	Time (ms)	Throughput (TFLOPs/s)
256	0.067	0.986	73.525	**0.0009**
512	0.135	0.988	71.857	**0.0018**
1024	0.270	0.989	71.873	**0.0037**
2048	0.540	0.990	68.303	**0.0079**
4096	1.080	0.990	70.723	**0.0152**
8192	2.160	0.990	72.749	**0.0296**
16384	4.320	0.990	73.172	**0.0590**
32768	8.640	0.990	74.421	**0.1160**

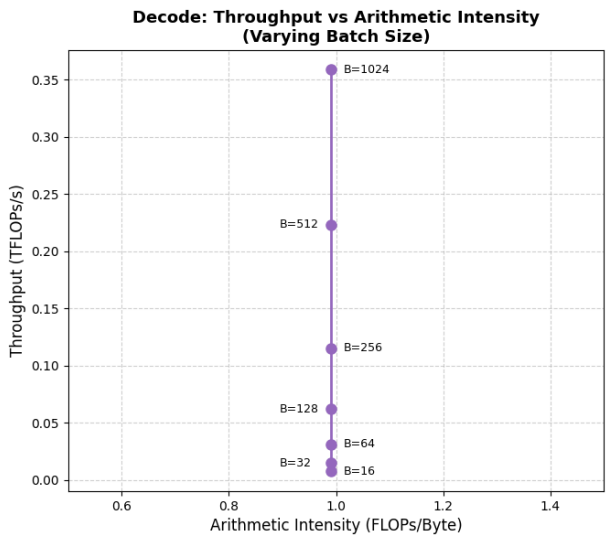


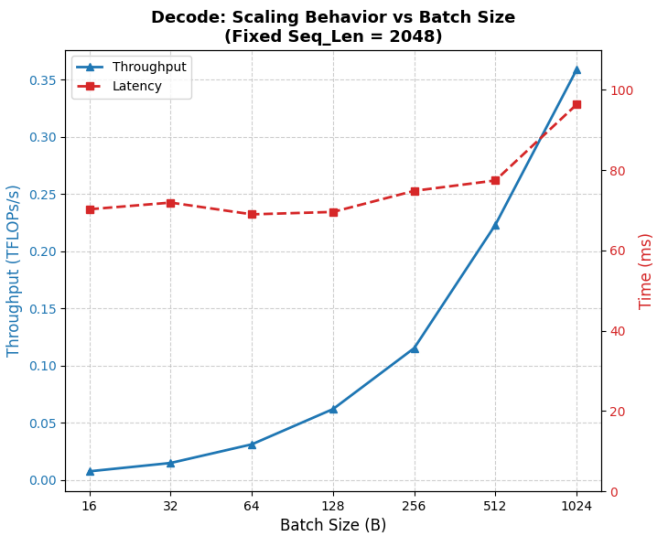


Explanation:
The execution time stays almost constant – which ind

Q11.

Batch (B)	Compute (GFLOPs)	AI (FLOPs/Byte)	Time (ms)	Throughput (TFLOPs/s)
16	0.540	0.99	70.267	**0.0077**
32	1.080	0.99	71.926	**0.0150**
64	2.160	0.99	69.000	**0.0313**
128	4.320	0.99	69.600	**0.0621**
256	8.640	0.99	74.852	**0.1154**
512	17.281	0.99	77.435	**0.2232**
1024	34.561	0.99	96.347	**0.3587**





Explanation:
Arithmetic Intensity is independent of batch size.
Throughput increases with batch size.

The Vertical AI Line: Because Arithmetic Intensity never changes (~0.99), plotting Throughput vs. AI yields a straight vertical stack of points. You are restricted to the memory-bound slope of the Roofline model.

Throughput Scaling via Amortization: If the workload is so memory-bound, why does throughput increase from 0.007 to 0.358 TFLOPs/s?

- Between B=16 and B=128, the execution time is basically flat (~70 ms). This indicates the GPU is still starving for work, and the latency is entirely consumed by Python dispatch overhead and kernel launch times.
- By batching up to 1024, you are forcing the GPU to load ~17 GB of KV Cache in a single pass. The framework overhead finally becomes amortized over a massive payload, driving the throughput metric up.

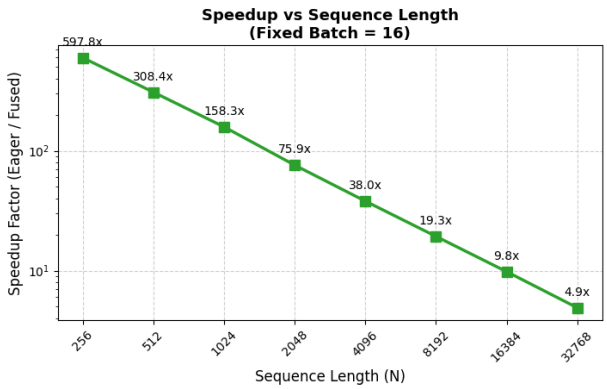
The Latency Rise: Notice that at B=256 through B=1024, the latency finally begins to climb (from ~74 ms up to ~96 ms). This is the exact moment the sheer volume of memory transfers begins to legitimately saturate the GPU's HBM bandwidth.

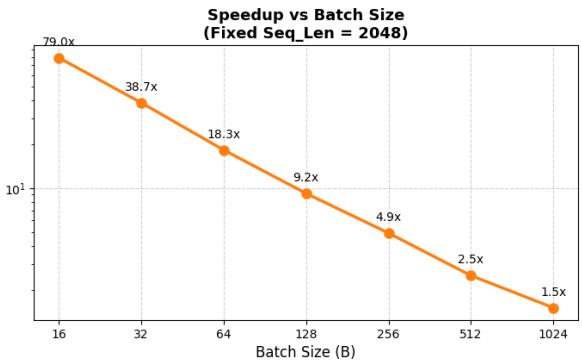
Q12.

Seq Len (N)	Eager (ms)	Fused (ms)	Speedup
256	73.525	0.123	**597.8x**
512	71.857	0.233	**308.4x**
1024	71.873	0.454	**158.3x**
2048	68.303	0.900	**75.9x**
4096	70.723	1.862	**38.0x**
8192	72.749	3.773	**19.3x**
16384	73.172	7.481	**9.8x**
32768	74.421	15.087	**4.9x**

Batch (B)	Eager (ms)	Fused (ms)	Speedup
16	70.267	0.890	**79.0x**
32	71.926	1.858	**38.7x**
64	69.000	3.770	**18.3x**
128	69.600	7.591	**9.2x**
256	74.852	15.315	**4.9x**

Batch (B)	Eager (ms)	Fused (ms)	Speedup
512	77.435	31.171	**2.5x**
1024	96.347	63.273	**1.5x**





Explanation:

The Numerator (Eager Overhead): As established, the eager PyTorch latency is "flat" at ~70ms for most of these runs. It is not bound by the GPU's memory or math; it is entirely bottlenecked by the sequential Python kernel launch overheads.

The Denominator (Fused Linearity): By fusing the operations, FlashAttention bypasses the framework overhead. You can see this because the Fused time perfectly doubles every time the workload doubles (0.12 to 0.23 to 0.45 to 0.90 ms). It is executing exactly at the hardware's memory bandwidth limit.

The 1/x Decay Curve: Because you are dividing a constant (Eager overhead) by a linearly increasing variable (Fused memory traffic), the resulting speedup mathematically decays proportionally to 1/x.